

OS_Process

#Process

Process : 실행 중인 프로그램. 시스템 내에서 작업 단위(unit)로서 작업을 수행함.

시스템은 많은 프로세스들을 가지고 있어 하나 이상의 CPU를 프로세스/스레드 간 **Multiplexing**함으로서 동시에 (concurrently) 실행함.

Multiplexer : 나가려는 곳은 하난데 나가려는 것들은 여러개

Multiplexing : CPU는 한 개인데 누구를 쓰게 할 것인가?

프로세스 : OS에서 실행 중인 프로그램.

스레드 : 프로세스 내에서 실행되는 작업 단위. 프로세스의 자원을 공유함.

OS는 프로세스 관리를 담당함.

User Process : 사용자가 실행하는 프로그램. 웹 브라우저, 문서 편집기

System Process : 운영체제가 자체적으로 실행하는 프로세스. 백그라운드 작업(장치 드라이버...)

멀티프로그래밍 : OS가 여러 프로세스를 메모리에 동시에 올려놓고 실행하는 기법. CPU 효율적 사용 가능.

Context Switching : 실행 중인 프로세스나 스레드에서 다른 프로세스나 스레드로 전환하는 과정을 의미함.

>CPU가 여러 작업을 동시에 실행하는 것처럼 보이게 함.

Job Scheduling	Macro	Long-term scheduling
CPU Scheduling	Micro	Short-term scheduling

#Process Management

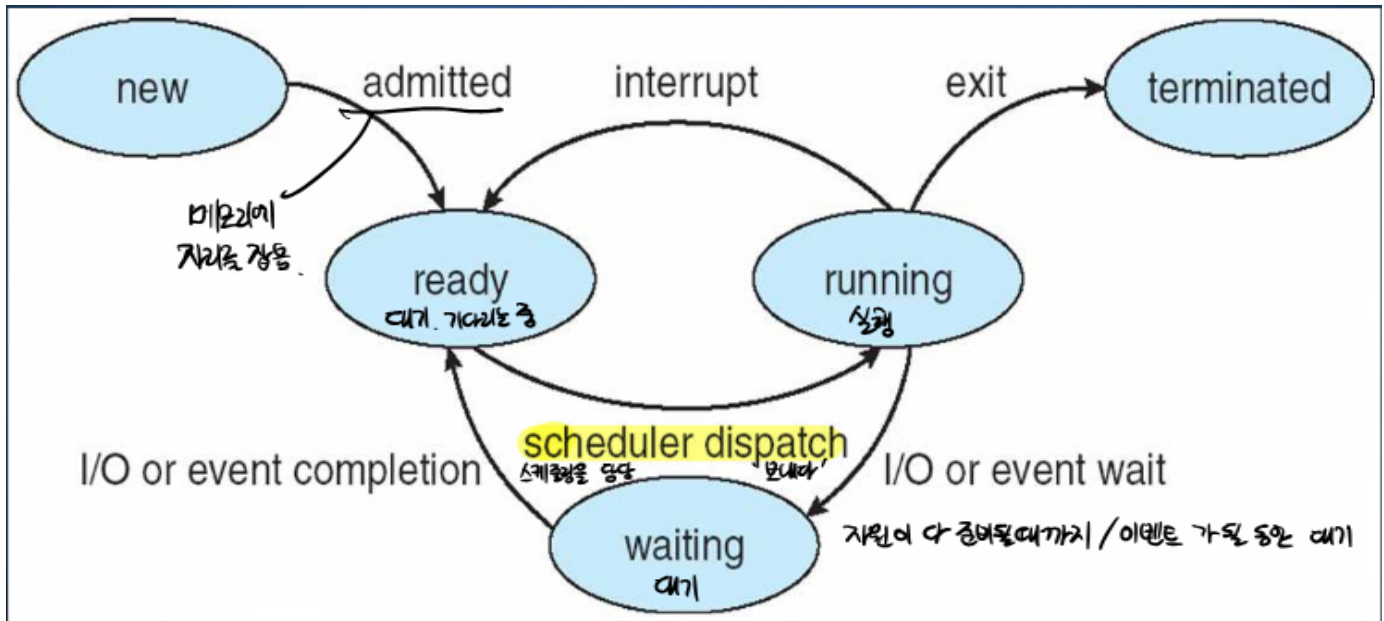
Create/Delete	프로세스 생성과 삭제	
Suspend/Resume	중단과 재개	실행되던 프로세스를 중단, 저장하고 다른 프로세스를 실행
Synchronize	동기화	여러 프로세스가 공유하는 자원에 동시에 접근할 때 충돌방지 등
Process Communication	통신	프로세스 간 데이터 교환 등
Deadlock Handling	데드락	교착 상태 처리. 무한 대기 상태

Deadlock vs bottleneck

Deadlock : 서로가 가진 자원을 기다리며 무한 대기 상태에 빠짐. 해결하지 않으면 영원히 멈춰서 대기.

<->Bottleneck : 병목현상; 자원이 집중되어서 생긴 문제. 언젠가는 해결됨.

Process state (transition) diagram ; 프로세스가 실행되면서 state가 바뀐다.



new : 새로 실행되고자 하는 프로그램

ready : 실행 준비가 됨

running : 프로세스 실행중

terminated : 끝남

waiting : 대기상태

admitted ; new -> ready

프로세스가 보조메모리에서 메인메모리로 올라왔다(자리를 잡음)! 실행 준비가 됨.

scheduler dispatch ; ready -> running

스케줄러(CPU 스케줄링을 담당하는 OS의 일정부분)가 스케줄링 알고리즘에 따라 보냄

exit ; running -> terminated

작업이 끝나면 프로세스 종료

I/O or event wait ; running -> waiting

I/O가 발생하거나 event가 일어나면 자원이 준비될 때까지 대기

I/O or event completion : waiting -> ready

I/O가 끝나거나 event가 끝나면 다시 ready 상태로

interrupt ; running -> ready

하던 일을 멈추고 다시 ready 상태로. 대표적인 예시가 context switching

PCB(Process Control Block)

PCB : 프로세스에 관한 모든 정보를 저장하는 저장소.

OS가 프로세스를 관리하기 위한 정보를 담아두는 곳.

모든 프로세스는 각 프로세스마다 PCB를 가지고 있음.

담겨있는 내용

프로세스의 상태 : ready, waiting, running etc...

Program Counter

> 프로세스에 대한 현재 정보

CPU register : accumulators, index register, stack pointers

> 프로세스에 대한 현재 정보

메모리에서의 프로세스

Text section - Program code

Data section - Global Variable

Heap - 런타임에 동적할당(Dynamic Allocate)된 데이터

Stack - 함수의 매개변수, 리턴된 주소, 로컬 변수같은 임시 데이터

>Heap과 Stack은 서로 공간을 공유함

Stack : Last In First Out

ex)함수를 여러번 호출하고 원래 코드로 돌아오려면 리턴될 주소를 기억하고 있어야 함.

나중에 호출한 함수로 되돌아가야함. > LIFO

Thread 스레드

스레드 : 독립적으로 실행될 수 있는 명령어들의 흐름(스트림)

OS는 각 스레드를 관리하고 스케줄링해서 CPU에서 실행할 수 있게 함.

*단일 스레드 프로세스(Single threaded Process)

하나의 스레드만을 포함하는 프로세스.

순차적 처리

*다중 스레드 프로세스(Multiple threaded Process)

여러 개의 스레드를 포함하는 프로세스

코드와 데이터 파일은 공유. 여러개의 스레드로 병렬(Parallel)로 실행될 수 있음. 효율성 성능 ▲

* 스레드 ID를 가진다

* 각각 다른 현재 상태(ready, running, waiting...)를 가진다 - PC, 레지스터 상태, 스택...

* 같은 프로세스에 속한 다른 스레드들과 코드, 데이터, 다른 OS자원을 공유함. ex)열린 파일

* 같은 주소 공간 내에서 동시에 여러 작업을 할 수 있음 - Parallel하게 실행되므로, 단일 프로세스 내에서 여러 작업을 동시에 수행할 수 있다. 각자 다른 실행 흐름을 가지고 있음.

3개의 스레드를 쓰지 않고 3개의 프로세스를 쓴다면, 같은 코드, 데이터, 파일이 같은데도 불구하고 프로세스마다 자원을 또 할당해야 함. (코드 가져오는데 시간걸리고, 메모리 할당해줘야함)

> 멀티스레딩을 하면 프로세스 간 communication이 불필요하다!

다중 스레드 프로세스의 예

웹 브라우저 - 하나의 스레드는 이미지나 텍스트를 표시하고, 나머지 스레드는 네트워크에서 데이터를 받아 올

웹 서버 - 여러 클라이언트들이 동시에(concurrently) 접근하고, 각각의 클라이언트들에게 서비스를 제공함

프로세스 생성 vs 멀티스레드 프로세스

시간 소모적(Time Consuming)

각각의 독립적인 프로세스를 생성하려면 메모리 공간, 시스템 자원이 더 필요. 복사하는데 시간이 듦.
스레드는 기존 프로세스의 주소 공간을 공유하기 때문에 추가적인 메모리 할당이 필요X
프로세스를 생성하는 것은 스레드를 생성하고 종료하는 것보다 시간이 많이 듦.

자원 집약적(Resource Intensive)

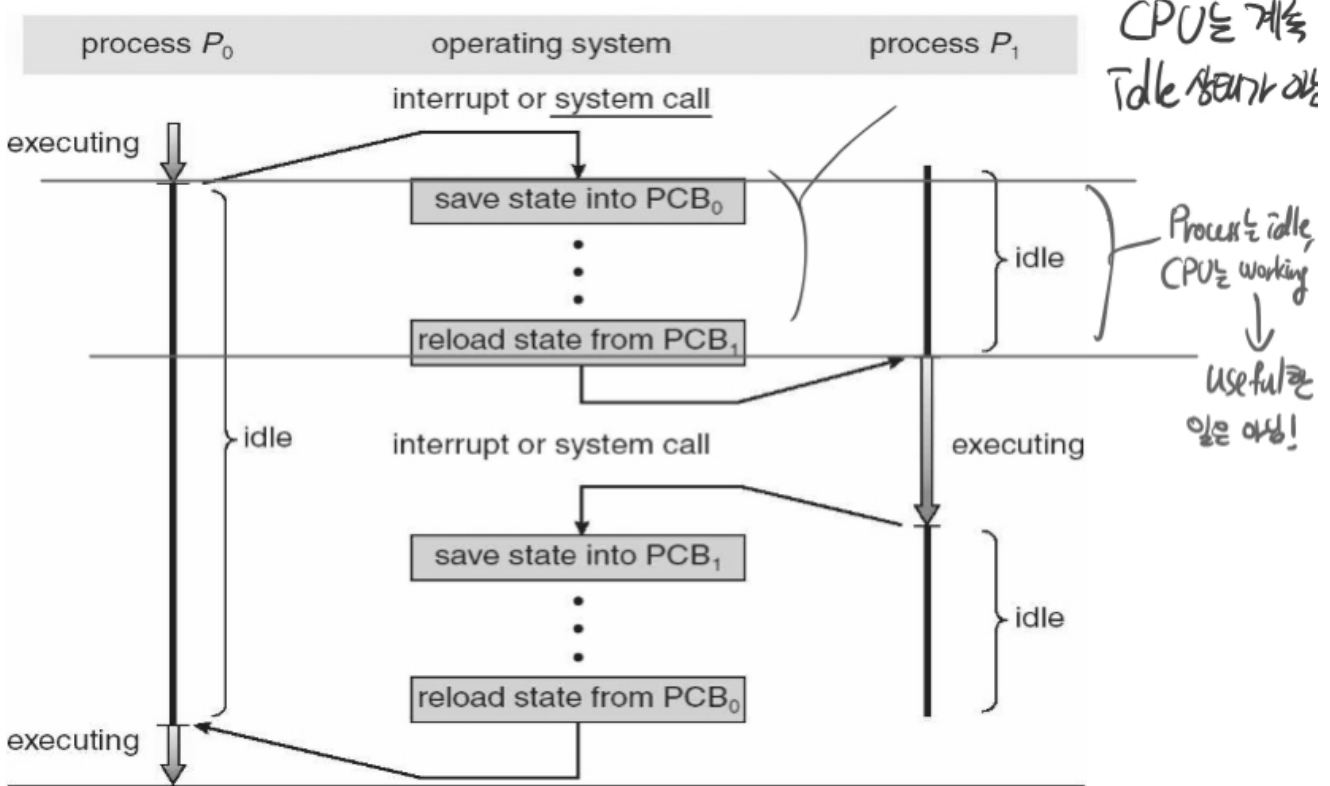
프로세스 : 각자 고유한 주소 공간을 가짐.
스레드 : 동일한 프로세스 내에서 공통 주소 공간을 공유함.

Context Switching - Process Switching(상황 교체; 프로세스를 바꾸는 것)

CPU를 사용하고 있는 프로세스를 다른 프로세스로 바꾸는 것
Context~~Current State

- * 프로세스는 동시에(Concurrently) 실행되며, CPU는 멀티플렉싱을 함.
- * 프로세스가 전환될 때, 시스템은 **현재 프로세스의 상태를 저장**하고, **새 프로세스의 저장된 상태**를 불러옴
- * 프로세스의 context는 PCB에 저장됨
- * Context Switch하는 것은 부담된다(overhead) - Save&Reload. CPU 입장에서는 유용한 일이 아님.
(idle도 아님)

Context Switch를 당하는 프로세스는 Status가 Running -> Ready로 바뀐다(Interrupt) ★NOT Waiting★



Q) 왜 멀티프로그래밍을 하려고 할까?

A) CPU의 Idle time을 줄여서 CPU의 작업처리율(throughput)을 높여 단위시간 안에 많은 일을 빠르게 처리할 수 있음.

CPU bound job process : CPU 사용이 주인 process

I/O bound job process : I/O가 주인 process

>> 둘이 적절히 섞이면 작업 속도가 빨라짐

Swapping

- * 메모리에서 프로세스를 제거하는 것 - Swap out
- * 메모리에 다시 불러오고 중단된 지점에서 계속 실행하는 것 - Swap in

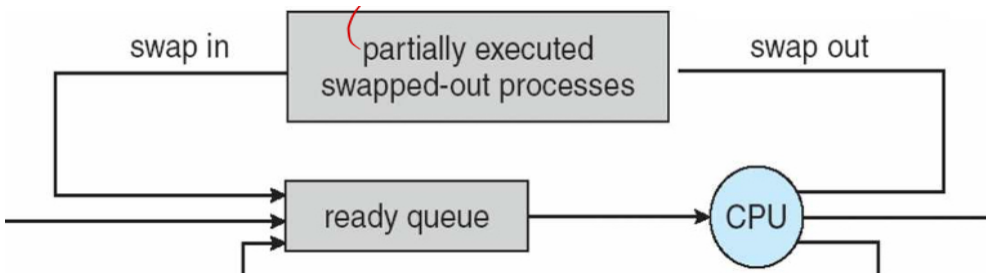
swapping이 필요한 이유(필요할 때)

- * 프로세스 믹스를 향상하기 위해 - CPU bound vs I/O bound 프로세스 간의 균형을 맞춰 자원
- * 가용 메모리 초과 상황에 대해 대처하기 위해

#starvation : 특정 프로세스나 스레드가 필요한 자원을 할당받지 못하여 무한정 기다리게 되는 상황

Q) Swapping이 많이 일어나면 왜 바람직하지 않을까?

A) Swapping 하는 동안에는 메인메모리와 보조메모리간 읽고 쓰는 시간이 오래 걸림



System Call

- * OS에서 제공하는 서비스에 대한 인터페이스를 제공
- * 대부분의 프로그램은 직접 시스템콜에 접근해 사용하기보다는, API를 통해서 시스템 콜에 접근
API는 시스템 콜을 감싸고 사용하기 쉬운 형태로 제공됨. 개발자는 시스템 콜의 세부 사항 신경X
- * API는 개발자가 사용할 수 있는 **함수들의 집합**을 정의함. 각 함수는 특정 작업을 수행, **매개변수** 포함
- * API를 구성하는 함수들은 개발자를 대신해서 실제 시스템 콜을 호출함.
>> API의 장점 : 추상화와 이식성

프로그램이 OS의 기능을 호출하면, 유저모드에서 커널모드로 전환됨.
시스템 콜은 유저모드와 커널모드 간의 인터페이스

API(Application Program Interface) :

function들의 집합

OS function들을 몰라도 기능 사용 가능.

추상화가 되어있어 사용자는 쉽게 기능을 사용 가능.

커널단에 변화가 있어도 유저는 상관하지 않아도 됨.

Porting 작업이 쉽다! Portability(이식성)가 좋다. 다른OS에도 적용하기 쉬움.

Q) 왜 시스템 콜(direct access) 대신 API를 쓰는가?

A) 프로그램의 이식성. OS의 디테일한 부분이 개발자로부터 숨겨져있음(추상화되어있음)

Deadlock 데드락

둘 이상의 프로세스나 스레드가 서로가 갖고 있는 자원을 기다리며 진행하지 못하는 상태

아래 4가지 필요 조건이 **동시에** 충족될 때 발생합니다.

=> 한가지라도 만족하지 않으면 deadlock 아님.

- * Mutual Exclusion 상호배타; 자원의 **독점적** 사용 (나눠 쓸 수 없음)
- * Hold and wait 자기 자원은 가지고 있으면서 대기 중
- * No **preemption** 강제로 뺏지 않음(비선점)
- * Circular wait 순환적으로 대기

데드락을 해결하는 방법

1. Prevention (방지) : 아예 안 일어나게 함. 4조건중에 하나라도 충족 못 시키게 함.
2. Avoidance (회피) : 일어날 거 같으면 자원 할당을 안 해줌.
3. Detection & Resolution : wait-for-graph에서 cycle이 있으면 그중 하나를 Victim을 정하고 kill 함.

Livelock 라이브락

프로세스는 계속해서 반응하지만 진행할 수 없는 상태. 서로 양보하려고 하면서 진행이 안 되는 상황.

라이브락으로 인해서 **Starvation**이 일어남.

해결하지 않으면 영원히 끝나지 않음.

Bottleneck 병목현상

특정 처리에서 자원이 부족하거나 느릴 때 발생.

가만히 두어도 언젠가는 해결됨.

OS_Scheduling

sequential execution : 순차처리

concurrent execution : 동시처리

Process 복습

멀티프로그래밍(concurrent)을 통해서 CPU의 사용 효율을 최대화함.

여러개의 프로세스가 한 메모리 안에 동시에 있음.

CPU-I/O Burst(CPU에서 실행되는 시간) Cycle

프로세스 실행은 CPU 실행과 I/O wait으로 구성됨

일반적으로 I/O-bound program은 짧고 많은 CPU burst를 가짐

CPU-bound program은 길고 적은 CPU burst를 가짐

>> 적절한 CPU 알고리즘을 고르는 것은 중요하다!

CPU scheduler

OS에서 CPU 스케줄링을 담당하는 코드 세그먼트.

* 메모리에 있는 프로세스 중에서 실행하도록 준비가 된 것 중 하나를 선택하고 CPU에 할당함.

* Queue에 저장된 레코드들은 일반적으로 PCB들이다. First In First Out

* CPU 스케줄러는 자주 호출된다. b.o. 여러 프로세스가 공정하게 CPU를 사용할 수 있게 하도록.

* 빨라야 한다. 실행 사이의 짧은 시간 때문에

ex) 스케줄러가 프로세스를 선택하는데 10ms가 걸리고, 이 프로세스가 100ms동안 실행된다면, 전체 시간 110ms 중 10ms는 스케줄링에 사용됨. 따라서 $10/(100+10) =$ CPU의 9%는 버려짐(프로세스 실행에 사용X)
분자가 작아질수록 Context Switching 하는 시간이 줄어듦.

스케줄링 결정이 내려지는 주요 상황들

* running -> waiting ex) I/O wait

* running -> ready ex) Expiration of time slice(Round robin). Interrupt, Context Switching

* waiting -> ready ★ ex) I/O 종료

>> Preemptive Scheduling. Ready에 새로운 것이 들어오면 우선순위를 update 해야 함
현재 프로세스를 계속 실행할지, 새로운 프로세스를 실행할지

* terminate

스케줄링 알고리즘

Nonpreemptive - FIFO

프로세스가 놓아줄때까지 프로세스가 CPU를 가지고 있음

Preemptive - Round Robin

강제로 CPU를 뺏음

Q) Shortest job first은 Nonpreemptive인가? Preemptive인가?

A) 둘다 아님! 정하기 나름. 둘다 가능함.

Dispatcher 디스패처

디스패처 : 단기 스케줄러가 선택한 프로세스에게 CPU 제어를 넘겨주는 역할 (진짜 CPU를 사용하게 함)

Short-term scheduler : CPU 스케줄러

Long-term scheduler : Job 스케줄러 job을 골라냄

디스패처의 역할

Context Switching

Switching to user mode from kernel mode 유저모드로 전환함.(디스패처가 OS라 원래 커널모드)

프로그램 실행 위치로 점프해서 재시작 (PC 값을 참고해서 예전에 실행되던 위치로)

디스패치 레이턴시(Dispatch Latency)

디스패처가 프로세스를 중지하고 다른 프로세스를 실행하기까지 걸리는 시간.

context switching하는데 걸리는 시간+CPU 제어를 넘겨주는 걸리는 준비시간

짧을수록 좋음. > CPU가 유용한 일을 할 시간이 늘어남

Round Robin 라운드 로빈

- * 각각의 프로세스가 CPU시간의 작은 단위(time quantum/slice)를 얻음. 대략 10~100ms.
- * 시간이 지나면 프로세스가 뺏기고, ready queue의 끝으로 감.
- * n개의 프로세스가 있고, time quantum이 q이면, 한 프로세스는 최대 (n-1)q단위시간만큼 기다린다.
- * preemptive(뺏기는) 스케줄링이다.

성능이 time quantum의 크기에 따라서 크게 영향을 받음

q가 매우 크면 > FCFS(먼저 오는게 먼저 스케줄링)이 되버림.

q가 작으면 > 프로세스가 계속 뺏기고 context switching이 빈번해져 overhead(부담)가 증가.
실제로 실행 할 시간이 없음.
>> context switching에 비해서 충분히 커야 함.

실제로는 10~100ms가 적당함. b.o. context switching하는데 보통 10마이크로초(0.01ms)가 들기 때문.

q를 100ms로 하면 q=100ms이고 context switching 시간=0.01ms이므로

=> context switching에는 전체 시간 대비 0.01%를 차지

q를 1ms로 하면 q=1ms이고 context switching 시간=0.01ms이므로

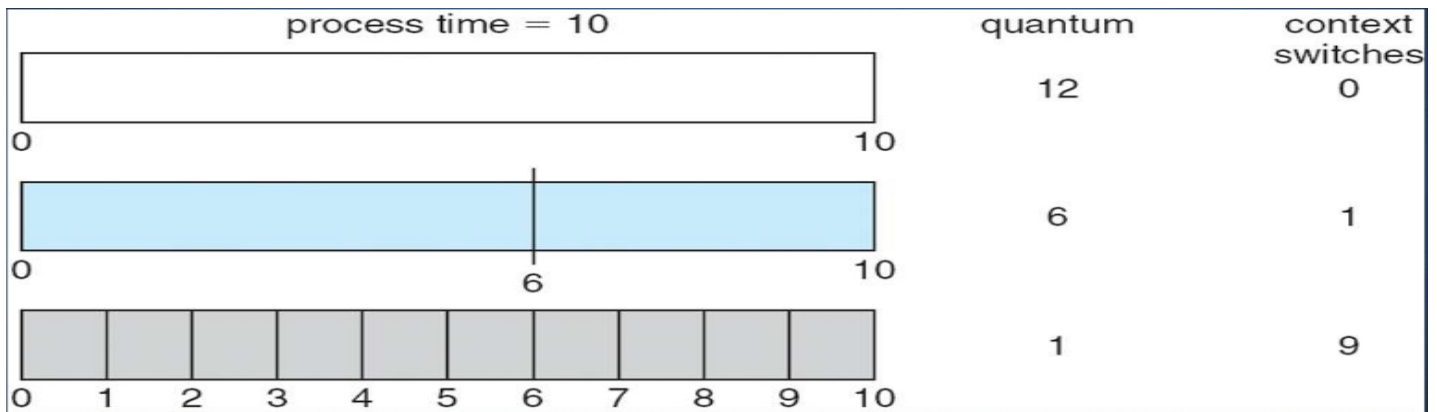
=> context switching에는 전체 대비 1%를 차지

>>>충분히 큰 값을 해야 시스템 성능을 저하시키지 않을 수 있음

• In practice, time quanta ranging from 10 to 100 ms, whereas time required for a context switch is typically less than 10microsecond(0.01ms)

Q)Time Quantum은 크면 클수록 좋은가? ==> X

Q)Time Quantum은 작으면 작을수록 좋은가? ==> X



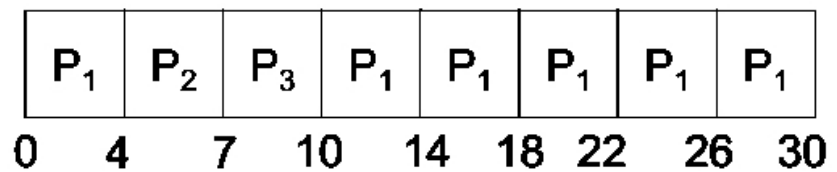
라운드로빈 예

Process Burst Time

P1 24

P2 3

P3 3



Quantum =4

Average waiting time은? $(p1이 기다린시간 + p2 + p3) / 3$
 $= [(10-4) + 4 + 7] / 3 = 5.66ms$

Turnaround Time은 Time quantum에 따라서 달라진다.

Turnaround time : 들어와서 나갈 때까지 걸린 시간; 기다리는시간 + execute한 시간

평균 Turnaround time은 time quantum이 커질수록 반드시 개선되는 것은 아니다.

>> 짧은 작업이 긴 작업 뒤에 계속 대기하게 만들어 평균 turnaround time을 늘릴 수 있음

q가 너무 짧으면 => 잦은 context switching > 오버헤드가 커짐

q가 너무 길면 => 짧은 일이 긴 일 뒤에서 긴 시간 기다려야 함. > 평균 turnaround시간 늘어남

대부분의 프로세스가 자신의 다음 CPU 버스트를 하나의 하나의 time quantum내에 끝난다면, 프로세스가 CPU를 할당받아 바로 작업을 종료 > queue로 가서 기다리지 않음, context switching 횟수 감소, 향상될 수 있음.

>> 일반적으로는 개선됨

Q) Time quantum이 클수록/작을수록 Turnaround time이 큰가/작은가?

A) 아님. 정비레도 아니고 반비례도 아님. q가 특정 값 이후로는 계속 같은 값이 나옴(한번에 끝나기 때문)

Ex 3) P1 P2 P3가 각각 10단위시간 걸림

Case 1 : time quantum = 10

P1 : 10(exe) = 10

P2 : 10(wait)+10(exe) = 20

P3 : 20(wait)+10(exe) = 30

Avg time qtm = $(10+20+30)/3 = 20$

Case 2 : time quantum = 1

p1 : 28

p2 : 29

p3 : 30

Avg time qtm = $(28+29+30)/3 = 29$

OS_Memory

Basics

프로그램은 반드시 메모리에 올라와있어야 한다.

메모리는 두 부분으로 나뉜다 - OS / Users

메모리 공간을 보호하기 위한 방법

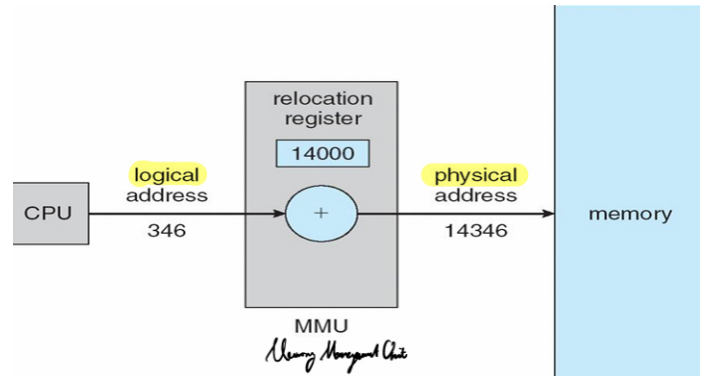
Base register : 특정 프로세스의 **제일 작은 physical address**

Limit register : 특정 프로세스의 **logical address**의 범위. 어디까지 접근할 수 있는지 정의. 크기

physical address : 실제 메모리 주소

logical address : CPU가 생각하는 주소

MMU



Memory Management Unit (하드웨어 장치)

logical에서 physical memory로 맵핑을 해줌

MMU가 각 프로세스의 mapping table을 참고해서 논리적 주소를 물리적 주소로 바꿔줌.

Contiguous allocation

Fixed Partition scheme : 파티션 크기를 고정된 크기로 일정하게 나눔. Internal Fragmentation 발생 가능.

한 파티션에 한 프로세스!

멀티프로그래밍의 개수가 **파티션의 개수**까지로 제한됨.

Internal Fragmentation : 메모리 공간 파편들이 생김

요즘은 거의 안쓰임

Variable partition scheme : 파티션 크기를 프로세스의 크기에 맞춰 동적으로 할당하는 방식. 필요한 만큼!

Fixed Partition에 비해서 효율적. External Fragmentation 발생 가능

메모리는 빈 공간에 할당됨. 어떤 빈공간에 넣을까?

First-fit : 첫 번째 수용할 수 있는 공간에(제일 먼저, 제일 위에)

+ scan 시간이 적게 걸림

- fragmentation이 발생할 수 있음

Best-fit : 가장 딱 들어맞는 곳에 할당(자기보다 같거나 큰 것 중 제일 작은 것)

+ 메모리를 효율적으로 사용할 수 있음. fragmentation 줄어듦

- scan 시간이 오래걸림

Worst-fit : 가장 큰 곳에 할당

+ 큰 빈 공간을 만들어 놓을 수 있음

- scan 시간이 오래걸림(best-fit과 비슷)

Fragmentation 문제

External Fragmentation : Variable Partition Scheme시

Internal Fragmentation : Fixed Partition Scheme시, Paging한 페이지 안에 남는 공간

Memory Compaction

External fragmentation을 줄임!

Fragmentation된 것을 모으는 것! 사용 중인 메모리를 한곳에

실행시(execution time)에만 compaction이 일어나고, **dynamic할 때만** relocation

어디로 움직일지 모르는 것==relocation이 dynamic하다.

>> compaction이 일어나면 CPU overhead가 발생시킬 수 있음. 자원이 필요함

Non-Continuous Allocation

Paging

Frames : Physical memory를 고정된 사이즈의 블록으로 나눈다.

Pages : Logical memory(프로세스)를 같은 고정된 사이즈의 블록으로 나눈다.

n개의 비어있는 프레임에 n 크기의 프로세스를 할당한다!

Frame이 최소Page 보다는 커야 함!

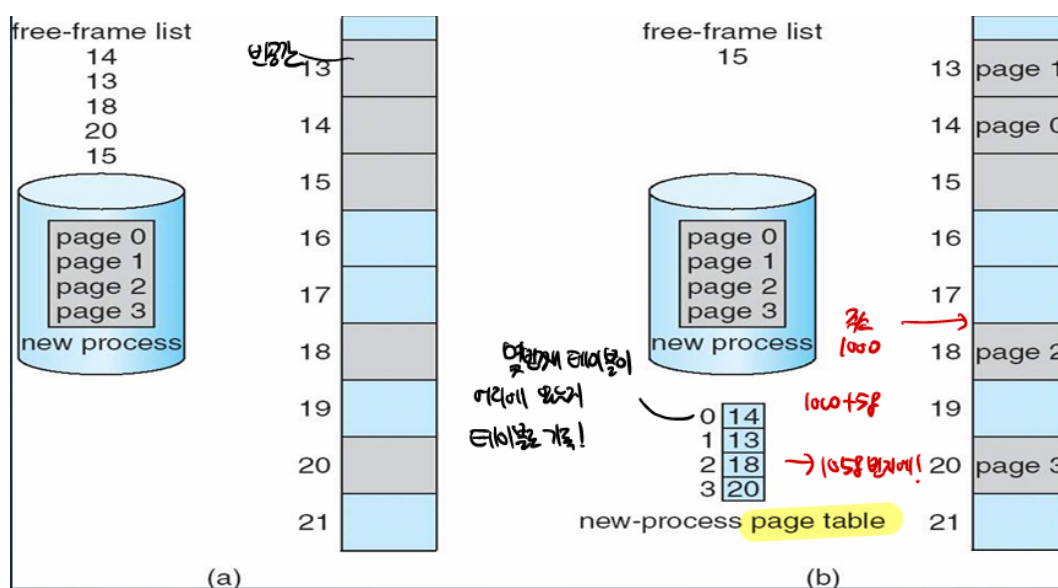
Paging에서는 External Fragmentation 발생하지 않는다!! 그래서 Paging을 쓰는 것

but, Internal Fragmentation은 발생한다.

프로세스에 해당하는 데이터 “다” 가져옴!

=> 필요한 것만 가져오는 건 demanding paging.

offset : 페이지 내에서의 위치. 페이지의 기준점에서 얼마나 떨어져 있는가? ~~ displacement



Page table : 실제로 데이터가 physical memory 어디에 위치가 있는지 접근할 수 있게 주소가 기록되어있는 테이블.

2진수로 생각하면

Page number(p) : 몇 번째 페이지인가

Page offset(d) : 얼마나 떨어져있는가

2^m 의 logical address space가있고, page size가 2^n 이라면

>>페이지는 총 $2^{(m-n)}$ 개만큼 있다!

따라서 2진수 주소에서는 맨 앞의 (m-n)bit만큼이 page number(p)를 나타내고
나머지 n bit만큼이 page offset(d)를 나타냄.

Demand Paging

필요할 때만 페이지를 메모리로 가져옴.

실행에 필요한 **일부 부분만** 메모리에 넣어 논다.

가상 메모리(virtual memory)의 개념을 적용함.

logical address space는 physical address space보다 클 수 있다.

> 실제로는 조금밖에 없지만 더 큰 용량도 **있는 것처럼** 사용할 수 있음

장점 : 프로그램 사이즈가 physical memory 크기에 제한받지 않는다.

더 많은 프로그램을 동시에(simultaneously)하게 실행할 수 있다.

CPU utilization과 throughput을 증가시킴

response time과 turnaround time을 감소시킴. > 빠름 b.o.

프로그램을 메모리에 load하거나 swap할 때 **I/O 작업 감소.**

>>**필요한 부분만** 메모리에 로드하기 때문에 디스크에서 메모리를 읽어오는 I/O 시간이 줄어듦!

Page Fault(페이지 폴트)

가져온 메모리 중에 필요한 페이지가 없는 것

valid-invalid bit 값이 현재 메모리에 필요한 페이지가 있는지를 나타냄

페이지 테이블의 valid-invalid bit 값이 0이면, 페이지 폴트가 일어난다.

>> 폴트가 일어나면 새로운 데이터를 보조메모리에서 가져와야 하므로 Interrupt 발생, 상태저장, I/O 함,
다시 불러와서 프로세스 실행

Thrashing(스레싱)

프로세스가 필요한 페이지가 없어서, **페이지 폴트가 너무 자주 일어나는 상태.**

프로세스가 page를 swap in, out 하느라고 바쁨 => CPU utilization이 low함

Pure Demanding Paging

: 필요하기 전까지 데이터를 절대 가져오지 않음. > 꼭 필요한 것만 가져옴

다수의 페이지 폴트 일어날 가능성.

>> Locality of Reference 특성을 잘 활용하면 폴트를 줄일 수 있다.

Pager(페이저) : 페이지들을 다루는 swapper

Swapper(스와퍼) : 프로세스 전체를 메모리에서 내보내고 불러오는 역할

Demand Paging VS Swapping

페이지 단위(Micro) 프로세스 단위(Macro)

Demand Paging에서의 2가지 Strategy

- * Frame Allocation : 프레임을 너무 많이 할당하면 멀티프로그래밍 ▼, 적게 할당하면 thrashing.
- * Page Replacement : 새로운 프로세스를 가져와야 할 때 누구를 내보낼 것인가?

Page Allocation Scheme (페이지 할당 계획)

Equal Allocation : 모든 페이지를 동일하게

Proportional Allocation : 프로세스 사이즈에 따라서. 비율에 따라서

Priority Allocation : 우선순위에 따라서

Page Replacement Strategy (페이지 대체 전략)

안 쓰는 것을 내보냄.

페이지 폴트를 최소한으로 일으키게 해야 함.

Modify bit(Dirty Bit)를 사용하여 “수정된” 페이지만 (보조메모리에) 쓰기. 1이면-> 쓰기 / 0이면-> 지우기

Global replacement : 교체할 프레임을 다른 프로세스의 프레임에서 고름

Local replacement : 교체할 프레임을 자기 프로세스 안에서 고름

FIFO : First In First Out

LRU : Least Recently Use => 일반적으로 성능이 좋음

Least Frequently Use

FIFO : First In First Out

- Belady's Anomaly : 프레임을 많이 줬는데 폴트가 더 늘어남.(항상 그런 것은 아님)

Reference page string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 일 때 프레임 3개보다 4개가 fault 자주,
처음 들어가는 것도 fault로 카운트하기

LRU : Least Recently Use

LRU

1	1	1	1	5
2	2	2	2	2
3	5	5	4	4
4	4	3	3	3

DB_1

Basics

데이터 모델링 : 현실세계에 있는 데이터를 잘 조직하는 행위

데이터 모델 : 데이터를 잘 표현하는 수단

Conceptual Model : 어떤 데이터가 중요한지, 데이터 간의 관계가 무엇인지, 사용자와 시스템 설계자 간의 의사소통을 목적으로 사용됨

ER(Entity-relationship) Model : Entity, Attribute, Relation으로 구성됨. Conceptual Model을 구체화하는데 사용

Relational Data Model : ER모델을 논리적 데이터베이스 설계로 변환 >> 2차원 테이블로 표현!

DBMS : 데이터베이스를 관리하는 시스템. Query Processing, Query Optimization 등등...

SQL : Structured Query Language

Interactive SQL : 대화형 SQL

Embedded SQL : Host Language 안에 끼어 들어간 SQL

File System Environment : APP마다 데이터를 각자 가지고 있음;

파일 시스템의 단점

프로그램-데이터 종속(dependence)가 문제

프로그램이 바뀌면 데이터도 바뀌어야 함.

Consistency 문제 : 한 곳에서 값을 바꿔도 다른쪽에서 바뀌지 않음

보안 문제 : 하나가 뚫리면 다른 것도 다 뚫림

Database system environment

프로그램-데이터 비종속(independene)

“통합”되어 저장되어 있고 데이터 “공유”

Data Model : 현실세계의 데이터를 표현하는 수단

Structure	구조	
Entity	개체	Entity는 Attribute를 가진다.
Relationship	관계	
Operation	명령, 작업	
Constraints	제약조건	

Relational DB Model

관계형 데이터베이스는 2차원 테이블이다!

“Relation”	테이블	Table	
“Attribute”	열	Columns	Fields
“Tuples”	행	Rows	Records
“degree”	차수	열(Attribute)이 몇 개 있는가?	
“cardinality”	계수	행(Tuple)이 몇 개 있는가?	

Domain : 속성이 가질 수 있는 값들의 집합

Domain Constraint : Domain의 제약조건(Constraint)

- 데이터타입, 범위, 제약조건...

Instance(Occurrence) : 틀에 대한 실제 들어있는 값 - 특정한 tuple

Schema : 개체들 간의 구조 + 제약조건

DB는 Schema를 표현할 수 있어야 한다.

Keys

각 Tuple(Record)를 unique하게 식별할 수 있는 Attribute(s) (Field).

; 하나 이상의 유일하게 식별할 수 있는 **attribute의 집합**

Superkey : 유일한 att의 집합

Candidate Key : 유일한 att의 집합 && 최소한의 att

Primary Key : NULL이 아닌 Key. Foreign Key를 참조.

Foreign Key : 외부의 attri Primary key에 의해서 참조당함.

(학번,이름,학과)에서 superkey와 candidate key의 조합은? 단, 학번은 unique하다.

Superkey : (학번), (학번, 이름), (학번, 학과), (학번, 이름, 학과)

Candidate : (학번) < b.o. 최소이기 때문

Q) 어떤 Relation에는 여러 개의 Candidate key가 있을 수 있는가?

A) 있음. (학번,이름,학과, 전번)이 있다고 하면, (학번), (전번)이 있을 수 있음.

** 모든 Relation은 최소한 하나의 Candidate key를 가진다.

** 모든 Relation은 최소한 하나의 superkey를 가진다.

** 모든 Relation은 반드시 Primary key가 존재해야 한다.

>> 한 Relation에는 하나의 Primary key가 있다.

** 모든 Primary Key는 Candidate Key/Superkey이다.

** Foreign key 는 여러 개가 있을 수 있다.

** foreign key는 여러 개의 attribute로 구성될 수 있다. (여러개를 조합하여 key로 사용 가능)

Q) 모든 Relation에는 Foreign key가 존재한다

A) X. 관계가 없으면 없을수도. (당연)

현실세계에 존재하는 의미(semantic)이 key인지 여부를 결정한다.

>>현실세계에서 '학번'이라는 것을 유일하게 했기에 key가 되는 것

★relation에서 값을 보고 판단하면 틀린 것!!★

Integrity(무결성)

4가지 무결성 **이텔릭 한 것은 Relational db라면 반드시 만족해야 함**

* User-defined integrity (business rules) : 조직마다의 룰

* Domain integrity :

* **Entity integrity** : 모든 Entity의 Primary Key의 값은 NULL이어서는 안 된다. rule no.1

* **Referential integrity** : Foreign key 값이 NULL이거나, 참조당하는 Primary key 안의 값이어야 한다.

Operation

3 basic operation

SELECT : 조건에 맞는 **Tuple**을 뽑음

JOIN : Relation끼리 연결

PROJECT : 조건에 맞는 **Attribute**를 뽑음

Q) 어떤 걸 먼저 할 것인가?

A) 그때그때 다름. 데이터의 크기나 빈도수에 따라서 달라진다. **Query Optimization**

System Catalog : optimization을 위해서 시스템은 통계치를 가지고 있음. 메타데이터.

Natural JOIN : 중복된 것 제외, 값이 같은 것들끼리 값 생성

$A \bowtie B$ 공통 attribute를 기준을 결합

Cartesian PRODUCT : A*이면 단순히 곱해서 다 연결시키기. 무조건 연결! $3*4=12$

$A \times B$ A의 cardinality*B의 cardinality = 결과의 cardinality

>> JOIN 하는 데는 시간이 많이 듦. JOIN 전에 SELECT나 PROJECT를 하는 것이 효과적.

연산을 할 때 relation은 항상 사각형이어야 한다.

***Union Compatible** : 집합 연산을 하기위한 조건(Intersect/Union/Difference/Divide 모두 해당)

1. **Same Degree** : Attribute의 수가 일치해야 한다.

2. **Same Domain** : 같은 domain끼리 연산해야 한다. 데이터타입, 제약조건 등...

SQL(Structured Query Language)

Non-procedural : 비절차적인 언어임. What(궁금한 것)만 얘기하고 How를 이야기하지않음

DDL(Data Definition Language) : CREATE, DROP, ALTER

DML(Data Manipulation Language) : SELECT, INSERT, DELETE, UPDATE

DCL(Data Control Language) : GRANT, REVOKE, COMMIT, ROLLBACK

 권한부여 권한회수 커밋 롤백

SQL Definition 예시

```
CREATE TABLE PRODUCT(
```

```
P_CODE                  VARCHAR(10)   NOT NULL UNIQUE,
```

```
P_DESCROPT             VARCHAR(10)   NOT NULL,
```

```
P_INDATE                DATE                NOT NULL, .....
```

```
PRIMARY KEY(P_CODE)
```

```
FOREIGN KET(V_CODE) REFERENCES VENDOR
```

```
ON DELETE RESTRICT    // 레퍼런스된 항목(FOREIGN KEY)가 지워졌을 때   >> 참조무결성 보장
```

```
ON UPDATE CASCADE    // 레퍼런스된 항목(FOREIGN KEY)가 바뀌었을 때   >> 참조무결성 보장
```

```
NO ACTION(아무것도 안함, 금지)~~RESTRICT   /   SET NULL(NULL로), SET DEFAULT(기본값으로),  
CASCADE(PRIMARY를 따라서)
```

SQL 쿼리를 받으면 **Query Optimization**을 한 후에 결과 출력

Embedded SQL : 프로그램 안에 들어있는 SQL문 <-> Interactive SQL

NO SQL : Not only SQL. 2차원 배열인 SQL만가지고는 안 된다.

Big Data의 3V : Volume 크다, Velocity 빠르다, Variety 다양하다